מטלה מסכמת

מבנה נתונים

קבוצה 7 – מערכת ניהול סקי

מגישים :

שקד ישראל, מאיה דניאלי, שחר לנדו, רועי רנר, תומר קמינר

מערכת ניהול סקי – מפרט פרויקט

1. **הגדרת מבני נתונים**

א. **מבני נתונים עיקריים**

המערכת משתמשת בשלושה מבני נתונים עיקריים לניהול יעיל שיעורי סקי:

(1) עץ סקי AVL (עץ חיפוש מאוזן)

- מטרה: אחסון ראשי לקבוצות, תלמידים ומסלולים
- יתרונות: פעולות חיפוש, הוספת ומחיקה ב- O(log n)
- איזון עצמי: שומר על גובה אופטימלי לאחר כל פעולה

(2) תור עדיפויות (Min-Heap)

- מטרה: ניהול סדר סבבים ותורות תלמידים
- יתרונות: גישה ב- O(1) לסטודנט הבא, עדכונים ב- O(log n)
- תכונת הערימה: שומרת אוטומטית על סדר עדיפויות

(3) רשימה ממוינת (מערך דינמי)

- מטרה: שימור סדר סבבים בתוך הקבוצות
- יתרונות: גישה ב- O(1) לפי אינדקס, שמירה על סדר הכנסת פריטים.
- חסכונית בזיכרון: הקצאת זיכרון רציפה

ב. **מבני נתונים תומכים:**

(1) מונה גודל

- מטרה: פעולות ספירה ב- O(1)
- מימוש: מוני שלמים שמתעדכנים בכל פעולה
- יתרונות: מונע מעבר על כל העץ (O(n)) לצורך ספירה

2. **מבני נתונים וקשרים - מערכות יחסיי הישויות בפרויקט**

א. עץ קבוצות ↔ עץ תלמידים: קשר רבים־לרבים דרך מזהי קבוצות

ב. עץ קבוצות ↔ עץ מסלולים: קשר רבים־לאחד דרך שיוך למסלול

ג. קבוצת סקי ↔ תור עדיפויות: קשר אחד־לאחד לניהול סבבים

ד. קבוצת סקי ↔ רשימה ממוינת: קשר אח ד־לאחד לשימור סדר תלמידים

3. **שדות נתונים ומפתחות (IDs)**

א. מחלקת תלמיד (Student)

```
class Student:
    stud_id: str          # מפתח ראשי – מזהה תלמיד ייחודי
    first_name: str       # שם פרטי
    last_name: str        # שם משפחה
    group_id: str         # מפתח זר – מקשר לקבוצת סקי
    group_ord: int        # מספר סדר בקבוצה (…1,2,3)
```

פרטי שדות:

- **stud_id:** מחרוזת – מזהה ייחודי לכל תלמיד.
- **first_name:** שם פרטי, אורך 1-50 תווים
- **last_name:** שם משפחה, אורך 1-50 תווים.
- **group_id:** מזהה הקבוצה אליה שייך התלמיד.
- **group_ord:** מספר שלם, סדרו של התלמיד בקבוצה.

ב. מחלקת קבוצת סקי (SkiGroup)

```
class SkiGroup:
    id: str                    # מפתח ראשי – מזהה קבוצה ייחודי
    tree: AVLTree              # עם תלמידי הקבוצה AVL עץ
    students_ordered: list     # רשימה ממוינת לשמירת סדר
    next_order: int            # המספר הסדרתי הבא
    current_round: int         # מספר הסבב הנוכחי
    assigned_track: SkiRoute   # מסלול מוקצה
    round_queue: PriorityQueue # תור עדיפויות לניהול סבב
```

פרטי שדות:

- **id:** מחרוזת, מזהה ייחודי לקבוצה.
- **tree:** עץ AVL עם תלמידי הקבוצה.
- **students_ordered:** רשימה לשימור סדר התלמידים.
- **next_order:** מונה אוטומטי לסדר הבא.
- **current_round:** מספר הסבב הנוכחי.
- **assigned_track:** מסלול סקי משויך.
- **round_queue:** תור עדיפויות לניהול תורות.

ג. מחלקת מסלול סקי (SkiRoute)

```
class SkiRoute:
    level: int            # מפתח ראשי – רמת קושי (1–5)
    name: str             # שם ותיאור המסלול
    max_students: int     # מספר תלמידים מקסימלי
    description: str      # פרטי המסלול
```

פרטי שדות:

- **level:** מספר שלם, רמת קושי (1=מתחילים, 5=מתקדמים).
- **name:** שם המסלול.
- **max_students:** מספר תלמידים מירבי.
- **description:** תיאור המסלול.

ד. **מחלקת סבב (Round)**

```
class Round:
    round_number: int     # מספר סבב
    student: Student      # הפניה לתלמיד
    score: float          # ציון (0–100)
    completed: bool       # האם הסבב הושלם
    priority: int         # עדיפות בתור
```

פרטי שדות:

- **round_number:** מספר שלם, מזהה הסבב.
- **student:** אובייקט תלמיד המשתף בסבב.
- **score:** ציון ביצוע.
- **compeleted:** האם הסבב הסתיים.
- **priority**: עדיפות בתור (לפי הסדר בקבוצה)

4. **<u>פונקציות ופעולות מערכת</u>**

א. **פונקציות ניהול עיקריות**

- **ניהול תלמידים**

(1) **הוספת תלמיד** - הוספת תלמיד חדש למערכת

(2) **מחיקת תלמיד** - הסרת תלמיד מהמערכת

(3) **שיוך תלמיד לקבוצה** - העברת תלמיד בין קבוצות

- **ניהול קבוצות**

(4) **יצירת קבוצה** - יצירת קבוצת סקי חדשה

(5) **שיוך מסלול לקבוצה** – הקצאת רמת קושי לקבוצה

- **ניהול סבבים**

(6) **התחלת סבב קבוצה** – פתיחת סבב חדש לקבוצה

(7) **קבלת תלמיד הבא** – שליפת התמיד הבא לגישה

(8) **רישום ניקוד סבב**- שמירת הוצאות הסבב עם ציון

ב. **פונקציות תצוגה ומידע**

(9) **הצגת מספר תלמידים בקבוצה**

(10)**הצגת תלמידי קבוצה**

(11)**הצגת כל הסבבים** - מבט הר על כל הקבוצות

(12)**סיכום מערכת** – סקירה כוללת

ג. **פעולות מתקדמות**

(13)**הסרת תלמיד מסבב** - ביטול השתתפות תלמיד בסבב

(14)**הצגת סדר תלמידי קבוצה**

5. **<u>מימוש מיון</u>**

א. **פונקציית מיון מותאמת**

אלגוריתם מיון מותאם שומר על סדר התלמידים לאחר הסרה.

סוג אלגוריתם: מיספור רציף

מטרה: שמירה על סדר תורות הוגן לאחר מחיקת תלמיד

ב. מיון בתור עדיפויות

תור העדיפויות משתמש בערימת מינימום לשמירה אוטומטית על סדר.

6. ממשקי מבני נתונים

א. ממשק עץ AVL

```
class AVLTree:
    def insert(data) -> AVLNode        # O(log n) – הוספת איבר
    def delete(data) -> bool           # O(log n) – מחיקת איבר
    def search(data) -> AVLNode        # O(log n) – חיפוש איבר
    def get_size() -> int              # O(1) – גודל העץ
    def is_empty() -> bool             # O(1) – בדיקה אם ריק
    def check_balance() -> bool        # O(n) – בדיקת איזון
```

ב. ממשק תור עדיפויות

```
class PriorityQueue:
    def enqueue(item)                  # O(log n) – הכנסה לתור
    def dequeue() -> item              # O(log n) – שליפת איבר
    def peek() -> item                 # O(1) – צפייה באיבר הבא
    def get_size() -> int              # O(1) – גודל התור
    def is_empty() -> bool             # O(1) – בדיקה אם ריק
    def remove(item) -> bool           # O(n) – הסרה של איבר
```

ג. ממשק קבוצת סקי

```
class SkiGroup:
    def add_student(student) -> bool          # O(log n) – הוספת תלמיד
    def remove_student(stud_id) -> bool       # O(log n) – מחיקת תלמיד
    def search_student(stud_id) -> Student    # O(log n) – חיפוש תלמיד
    def get_student_count() -> int            # O(1) – מספר תלמידים בקבוצה
    def start_new_round() -> bool             # O(n) – התחלת סבב חדש
    def get_next_student() -> Student         # O(1) – קבלת תלמיד הבא
```

7. הוכחת נכונות

א. נכונות עץ AVL

אינווריאנט: אחרי כל פעולה. נשמרת תכנות איזון של עץ AVL.

ב. נכונות תור עדיפויות

אינווריאנט: נשמרת תכונת הערימה אחרי כל פעולה.

ג. נכונות סדר סבבים

אינווריאנט: תלמידים תמיד גולשים לפי סדרם המקורי בקבוצה.

למעשה הנכונות נובעת מנכונות המבני נתונים עצמם בהם בחרנו להשתמש,
והבדיקות שלנו מראות שהקוד שלנו מממש אותם בצורה תקינה ויעילה.

8. ניתוח ביצועים

א. סיכום סיבוכיות זמן

פונקציה	מבנה נתונים בשימוש	פעולות	מטרה	סיבוכיות זמן
add_student	עץ תלמידים (AVL Tree) + עץ הקבוצה (AVL Tree)+ רשימת תלמידי הקבוצה (List)	**insert()** + **insert()** + **append()**	הוספה לעץ המערכת + לעץ הקבוצה + לרשימת הסדר	**O(log n+m)**
student_remove	עץ תלמידים (AVL Tree) + עץ הקבוצה (AVL Tree) + רשימת תלמידי הקבוצה (List) + תור עדיפויות של הקבוצה (Priority Queue)	**delete()** + **delete()** + **remove()** + **remove()**	הסרה מהמערכת + מהעץ של הקבוצה + מהרשימה + מתור הסבב	**O(log n+m)**
assign_student_to_group	עץ תלמידים (AVL Tree) + עץ הקבוצה (AVL Tree) + עץ הקבוצה (AVL Tree) + רשימת תלמידי הקבוצה (List)	**search()** + **search()** + **insert()** + **append()**	מציאת תלמיד + מציאת קבוצה + הוספה לעץ הקבוצה + הוספה לרשימת הסדר	**O(log n+m)**

ב. פונקציות ניהול קבוצות

פונקציה	מבנה נתונים בשימוש	פעולות	מטרה	סיבוכיות זמן
create_group	עץ קבוצות (AVL Tree)	**insert()**	יצירת קבוצה חדשה במערכת	**O(log n)**
Assign_track_to_group	עץ קבוצות (AVL Tree) + עץ מסלולים (AVL Tree)	**search()** + **search()**	מציאת קבוצה + מציאת מסלול + שיוך	**O(log n)**

ג. פונקציות ניהול סבבים

פונקציה	מבנה נתונים בשימוש	פעולות	מטרה	סיבוכיות זמן
start_group_round	רשימת תלמידי הקבוצה (List) + תור עדיפויות (Queue Priority) + רשימת סבבים (List)	**מעבר על רשימה** + **enqueue()** + **append()**	יצירת סבבים לכל התלמידים + הוספה לתור + שמירה ברשימת סבבים	**O(mlog m)**
get_next_round_student	תור עדיפויות (Queue Priority)	**peek()**	קבלת התלמיד הבא ללא הסרה	**O(1)**
complete_student_round	תור עדיפויות (Queue Priority) + רשימת סבבים (List)	**remove()** +**חיפוש ברשימה**	הסרה מהתור + סימון הסבב כהושלם	O(m)

O(m)	הסרה מהתור + סיום סבב	**remove()** **+חיפוש ברשימה**	תור עדיפויות (Queue + Priority) רשימת סבבים (List)	**remove_student_from_round**

ד. פונקציות תצוגה ושאליתות

סיבוכיות זמן	מטרה	פעולות	מבנה נתונים בשימוש	פונקציה
O(m)	הצגת תלמידים לפי סדר	**מעבר על רשימה**	רשימת תלמידי הקבוצה (List)	**display_students_in_group**
O(1)	קבלת מספר תלמידים	**get_size()**	עץ הקבוצה (AVL Tree)	**display_group_ student_count**
O(n+m)	מעבר על כל הקבוצות והצגת סבבים	**מעבר על עץ + מעבר על רשימה**	עץ קבוצות (AVL Tree) + רשימת סבבים (List)	**display_all_rounds**
O(n+m)	סיכום מערכת כולל + פרטי קבוצות	**get_size()** + **traversal()** + **get_size()**	עץ קבוצות (AVL Tree) + עץ תלמידים (AVL Tree) + עץ מסלולים (AVL Tree) + עצי קבוצות	**get_system_summary**

ה. פונקציות חיפוש / איתור

פונקציה	מבנה נתונים בשימוש	פעולות	מטרה	סיבוכיות זמן
_find_group	עץ קבוצות (AVL Tree)	**search()**	מציאת קבוצה לפי מזהה	**O(log n)**
_find_student	עץ תלמידים (AVL Tree)	**search()**	מציאת תלמיד לפי מזהה	**O(log n)**
_find_track	עץ מסלולים (AVL Tree)	**search()**	מציאת מסלול לפי רמת קושי	**O(log n)**
min_student	עץ תלמידים (AVL Tree)	**search()**	מציאת מספר סידורי מינימלי בקבוצה	**O(log n)**

9. בדיקות ואימות

א. נתוני בדיקה

כולל נתוני בדיקה ל-5 קבוצות, 25 תלמידים, ו-10 מסלולי סקי קוד הבדיקה נתון בקובץ test_system ופלט תוצאותיו בקובץ text_results.txt

ב. בדיקות פונקציה

כוללות פעולות רגילות, מקרי קצה, תנאי שגיאה ובדיקות ביצועים.

ג. תוצאות אימון

המערכת מצליחה להראות:

- שימוש נכון במבני נתונים
- ביצועים יעילים
- טיפול נכון בשגיאות

10. מסקנה

מערכת ניהול הסקי מממשת מבני נתונים אופטימליים לדרישות:

1. **עצים מאוזנים (AVL)** O(log n) לפעולות חיפוש/הוספה/מחיקה
2. **תורי עדיפויות** מאפשרים גישה מיידית לסטודנט הבא
3. **רשימות ממוינות** שומרות על סדר תורות הוגן
4. **מונה גודל** מספק ספירה מיידית